

# ASP.NET Core Backend Security Project – Full Roadmap

## Phase 1 — Project Setup & Data Ingestion

- 1 Create ASP.NET Core Web API
- 2 Configure EF Core
- 3 Import finance dataset into DB
- 4 Map dataset tables to EF Core entities
- 5 Add minimal tables: Users, AuthSessions or RefreshTokens, AuditLogs
- 6 Configure Serilog logging

## Phase 2 — User Registration & Login

- 1 Create User entity
- 2 Implement registration endpoint
- 3 Hash passwords securely
- 4 Enforce unique email constraint
- 5 Implement login endpoint
- 6 Track failed login attempts and last login time
- 7 Temporary account lockout after N failures

## Phase 3 — Authentication System

- 1 Session-based auth: session table, cookies, rotation, logout, expiration
- 2 Token-based auth: access + refresh tokens
- 3 Store hashed refresh tokens
- 4 Rotate refresh tokens on use
- 5 Detect token reuse and revoke all sessions

## Phase 4 — Authorization & Secure Data Access

- 1 Add authentication middleware
- 2 Create authorization policies
- 3 Protect endpoints
- 4 Implement row-level ownership checks
- 5 Prevent IDOR attacks
- 6 Add admin-only endpoints

## **Phase 5 — Expense & Subscription APIs**

- 1 Transaction listing endpoint
- 2 Filtering by date range, merchant, category
- 3 Aggregation: monthly totals and category breakdown
- 4 Subscription detection: same merchant, similar amount, recurring intervals

## **Phase 6 — Security Hardening**

- 1 Rate limiting (AspNetCoreRateLimit)
- 2 Input validation (FluentValidation)
- 3 Global exception handling
- 4 Prevent sensitive error leaks
- 5 Secure password reset flow (time-limited, single-use)
- 6 CSRF protection (if cookies used)

## **Phase 7 — Audit Logging & Monitoring**

- 1 Create audit log service
- 2 Log logins, failures, password changes, permission denials
- 3 Expose admin audit endpoints
- 4 Add suspicious activity flags

## **Phase 8 — Advanced / Hard Mode**

- 1 Login from new device detection
- 2 IP / geo anomaly detection
- 3 Encrypted columns using EF Core value converters
- 4 Soft deletes with recovery
- 5 Background hosted services
- 6 API versioning
- 7 GDPR-style data export

## **Attack Scenarios to Test**

- 1 Brute-force login
- 2 Session fixation
- 3 Token replay
- 4 IDOR access attempts
- 5 SQL injection attempts
- 6 CSRF attacks

## 7 Privilege escalation

### **Why This Is a Strong .NET Project**

- 1 Build enterprise-grade ASP.NET APIs
- 2 Use Identity correctly
- 3 Handle real security concerns
- 4 Work with legacy databases
- 5 Think like a backend engineer